IMT Mines Alès
École Mines-Télécom

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

Prompting & LangChain Vocabulary

16/10/2026

Komlan AYITE

R&D Engineer in AI and Data Science – IMT Mines Alès / CERIS

Outline

1. 1. From CM1 to CM2

1.1. What you built on Tuesday

1.2. Roadmap for today

2. 2. Calling an LLM

2.1. invoke / stream / batch

2.2. Message roles

3. 3. Prompting Strategies

3.1. Zero-shot and few-shot

3.2. Chain-of-Thought

3.3. ReAct prompting

4. 4. Structured Outputs

4.1. Why raw text is not enough

4.2. Pydantic schemas

5. 5. Memory & Embeddings

5.1. Short-term memory

5.2. Embeddings and semantic similarity

6. 6. Summary

6.1. 5 things to remember

6.2. Module roadmap

1. From CM1 to CM2

Where We Left Off

CM1 gave you the why

- LLM ≠ Agent
  stateless vs. stateful – one-shot vs. loop
- Four components
  LLM · Tools · Memory · Orchestrator
- ReAct = Reason + Act
  Perceive → Reason → Act → Observe
- Local models via Ollama
  `qwen2.5:3b` – no API key needed

CM2 gives you the vocabulary

- Three ways to call an LLM
  `invoke` · `stream` · `batch`
- Prompting strategies
  zero-shot · few-shot · chain-of-thought
- Structured outputs
  JSON mode · Pydantic · `with_structured_output`
- Short-term memory
  the messages list as agent state
- Embeddings & semantic similarity
  the bridge to TP2 and RAG

The thread: CM1 taught you *how* the loop works. CM2 gives you the *code vocabulary*. TD1 makes you *build* it.

Roadmap for Today

I. Calling an LLM (LangChain)

- `invoke` / `stream` / `batch`
- Message roles: System, Human, AI
- What the model actually receives

II. Prompting Strategies

- Zero-shot and few-shot
- Chain-of-Thought (CoT)
- ReAct prompting

III. Structured Outputs

- Why raw text is not enough
- JSON mode & Pydantic
- `with_structured_output`

IV. Memory & Embeddings

- Short-term memory = the messages list
- Embeddings as vectors
- Cosine similarity
- Bridge to TP2 (RAG)

Outcome of today

After this course you will know *what* each LangChain call does and *why* it is designed that way.

2. Calling an LLM

Three Ways to Call an LLM

invoke — synchronous call

```python
from langchain_ollama import ChatOllama
llm = ChatOllama(model="qwen2.5:3b", temperature=0)

response = llm.invoke("What is 3 + 5?")
print(response.content)    # "8"
```

stream — token by token

```python
for chunk in llm.stream("Explain ReAct briefly."):
    print(chunk.content, end="", flush=True)
# tokens arrive one by one -- good for UX
```

batch — many inputs at once

```python
responses = llm.batch([
    "Translate 'hello' to French.",
    "What is the capital of Togo?",
])
# returns a list of AIMessage objects
```

When to use each

- **invoke** — TD1 exercises, agents, any synchronous call
- **stream** — chatbots, streaming UI, long answers
- **batch** — offline processing, embedding datasets, evaluation

Key insight

The three methods share **identical input/output types**.

`invoke` returns `AIMessage`.
`stream` yields `AIMessageChunk`.
`batch` returns `list[AIMessage]`.

Same interface — swap freely.

Message Roles — What the Model Receives

The four message types

- `SystemMessage` — instructions to the model
  role, constraints, output format
- `HumanMessage` — the user's input
  the question or task
- `AIMessage` — the model's reply
  text content or `tool_calls`
- `ToolMessage` — tool result
  fed back so the model can read it

What the model actually sees

```
[SystemMessage]
"You are a maths tutor."

[HumanMessage]
"What is (3+5) * 12?"

[AIMessage + tool_calls]
{name: "add", args: {a:3, b:5}}

[ToolMessage]
"8"

[AIMessage + tool_calls]
{name: "multiply", args: {a:8, b:12}}

[ToolMessage]
"96"

[AIMessage]
"The answer is 96." ← done
```

3. Prompting Strategies

Zero-Shot vs. Few-Shot Prompting

Zero-shot — no examples

Give a task. No demonstration. The model uses its training.

```
system = SystemMessage("""
Classify the sentiment of the text.
Reply with one word:
positive, negative, or neutral.
""")
human = HumanMessage("This coffee is amazing!")
response = llm.invoke([system, human])
# "positive"
```

When to use: common tasks, well-trained models, simple output format.

Few-shot — 2–5 examples in the prompt

Show the pattern. The model imitates it.

```
system = SystemMessage("""
Classify sentiment. Examples:
  "I love it!" -> positive
  "It was terrible." -> negative
  "It was okay." -> neutral
Now classify the user's text.
""")
human = HumanMessage("The delivery was late.")
response = llm.invoke([system, human])
# "negative"
```

When to use: unusual format, domain-specific labels, small models that need more guidance.

The system prompt is your main lever. A precise system prompt beats a vague one every time — whether zero-shot or few-shot.

Wei et al., 2022 – arXiv:2201.11903

Chain-of-Thought (CoT) Prompting

Without CoT — direct answer

```
human = HumanMessage(
  "Roger has 5 tennis balls. "
  "He buys 2 cans of 3 balls each. "
  "How many balls does he have?"
)
# Small models often answer: "11" <- WRONG
```

With CoT — think step by step

```
system = SystemMessage(
  "Think step by step before answering."
)
# Model reasons:
# "Roger starts with 5 balls.
#  He buys 2 * 3 = 6 more.
#  5 + 6 = 11."
# -> "11"  <- correct AND verifiable
```

Why CoT works

- Forces the model to **externalise** its reasoning
- Each step **constrains** the next
- Intermediate steps are **verifiable**
- Errors become visible — and fixable

CoT → ReAct

CoT = reason in text.

ReAct = reason in text *and* act with tools.

The Thought–Action–Observation trace
is CoT with real-world grounding.

Wei et al. (arXiv:2201.11903)
Yao et al. (arXiv:2210.03629)

ReAct Prompting — Reason and Act

What the system prompt tells the model

```
system = SystemMessage("""
You are a helpful assistant.
You have access to tools.
Use them to answer accurately.

When you need information:
  1. Think which tool to call.
  2. Call it with the right arguments.
  3. Read the result.
  4. Continue until you can answer.

When done, give a clear final answer.
""")
```

Three roles of the system prompt

- **Persona** — who the model is
  "You are a maths tutor."
- **Tools** — when and how to use them
  "Use tools when you need real data."
- **Format** — what the output looks like
  "Reply in JSON. One sentence only."

The docstring IS the prompt.

The model reads your `@tool` docstring to decide *when* and *how* to call the tool.

Good: *"Add a and b. Returns float."*
Bad: *"addition"*

You practise this in TD1 Exercises 4–5.

4. Structured Outputs

Why Raw Text Is Not Enough

The problem with raw text

```
response = llm.invoke(
  "Extract name, age, city from: "
  "'Alice, 30, Paris'"
)
print(response.content)
# Could be any of:
# "Name: Alice, Age: 30, City: Paris"
# "{'name': 'Alice', 'age': 30}"
# "Alice is 30 and lives in Paris."
```

Problem: format varies between calls, models, and temperatures — unparseable.

with_structured_output

```
from pydantic import BaseModel, Field

class Person(BaseModel):
    name: str = Field(description="Full name")
    age:  int = Field(description="Age in years")
    city: str = Field(description="City")

structured_llm = llm.with_structured_output(Person)

person = structured_llm.invoke(
  "Extract from: 'Alice, 30, Paris'"
)
print(person.name)   # "Alice"
print(person.age)    # 30  (int!)
print(person.city)   # "Paris"
```

LangChain compiles the Pydantic schema into a JSON schema and injects it into the system prompt automatically. **The model returns valid JSON. LangChain gives you a Python object.**

Pydantic Schemas — Precise and Validated

A richer Pydantic model

```python
from pydantic import BaseModel, Field
from typing import List

class AgentEvaluation(BaseModel):
    """Evaluate an agent's response."""
    score: int = Field(ge=1, le=5,
        description="Quality score 1-5")
    rationale: str = Field(
        description="One-sentence justification")
    issues: List[str] = Field(default=[],
        description="Problems found")
    improved: str = Field(
        description="Suggested improvement")

eval_llm = llm.with_structured_output(AgentEvaluation)
result = eval_llm.invoke(
  "Evaluate: 'The capital of France is Lyon.'"
)
print(result.score)    # 1
print(result.issues)   # ["Paris is the capital"]
```

What Pydantic gives you

- **Validation** — wrong type raises error
- **Constraints** — `ge=1, le=5` enforced
- **Defaults** — optional fields handled
- **Python objects** — `result.score`, not `result["score"]`
- **Self-documenting** — `Field(description=...)` becomes the model's prompt

5. Memory & Embeddings

Memory & Embeddings

Short-Term Memory — The Messages List

Memory = the growing messages list

- Every LLM call receives the **full history**
- The model "remembers" by **re-reading** this list
- You append after every turn:
  Human → AI → Tool → AI → ...
- Limit: the **context window**
  32k–1M tokens depending on model

A minimal conversational loop

```python
history = [SystemMessage("You are a tutor.")]

def chat(user_input: str) -> str:
    history.append(HumanMessage(user_input))
    response = llm.invoke(history)
    history.append(response)    # AIMessage
    return response.content

print(chat("What is an embedding?"))
print(chat("Can you give an example?"))
# The model remembers the first question
```

Messages list in a ReAct agent

```
messages = [SystemMessage, HumanMessage]
--- step 1 ---
append AIMessage(tool_calls=[add])
append ToolMessage(content='8')
--- step 2 ---
append AIMessage(tool_calls=[multiply])
append ToolMessage(content='96')
--- step 3 ---
append AIMessage(content='The answer is 96.')
→ loop exits, return final answer
```

Short-term = context window (this session).

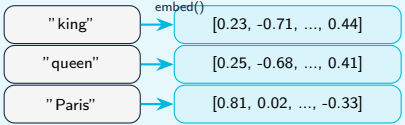
Long-term = external storage
(vector DB — ChromaDB, FAISS, Pinecone).

You build long-term memory in **TP2**.

Embeddings — Text as Vectors

What is an embedding?

An embedding model maps any text to a **vector of numbers** in a high-dimensional space (typically 384–3072 dimensions).



Key property: semantically similar texts have **similar vectors** — even different words.

Cosine similarity

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \, \|B\|} \in [-1, 1]$$

1 = identical 0 = unrelated -1 = opposite

In LangChain

```python
from langchain_ollama import OllamaEmbeddings
import numpy as np

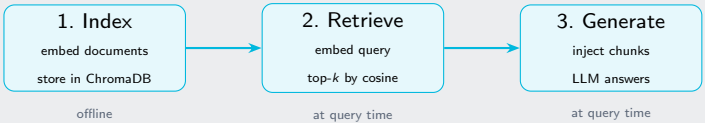
emb = OllamaEmbeddings(model="nomic-embed-text")
v1 = emb.embed_query("What is an agent?")
v2 = emb.embed_query("Define an AI agent.")
v3 = emb.embed_query("Today's weather?")

def cosine(a, b):
    a, b = np.array(a), np.array(b)
    return float(np.dot(a,b) /
                 (np.linalg.norm(a)*np.linalg.norm
                       (b)))

print(cosine(v1, v2))  # ~0.93  -- similar
print(cosine(v1, v3))  # ~0.51  -- unrelated
```

Embeddings — Bridge to RAG (TP2)

The RAG pipeline in three steps



Today you understand the retrieve step

- Query → embedding vector
- Compare to all stored document vectors
- Return top-k closest chunks (cosine)
- These chunks become the agent's context

In TP2 you build the full pipeline

- Chunk and embed a PDF
- Store in ChromaDB
- Write `retrieve(query)` as a `@tool`
- Plug it into your `react_agent()` from TP1 **zero changes to the loop**

6. Summary

Summary — 5 Things to Remember

1. **invoke / stream / batch** are interchangeable.
   Same types in, same types out.
2. **Message roles** define the conversation.
   System = instructions. Human = query. AI = reply. Tool = result.
3. **Zero-shot + good system prompt** wins.
   Few-shot when the model needs examples of the exact format.
4. **CoT → ReAct**.
   CoT externalises reasoning. ReAct adds tool calls as actions.
5. **Structured outputs** make LLMs machine-readable.
   Pydantic schema → JSON constraint → Python object.

For TD1 right now

```
ollama pull qwen2.5:3b
pip install langchain-ollama pydantic
```

Open `TD1_LangChain_Basics.ipynb`

What Comes Next

Session	Date	You will build
CM1 + TD1	Oct 13	The agentic paradigm · `@tool` · single tool call
CM2 + TD1 + TP1	**Oct 16**	**Prompting · structured output · full ReAct agent**
TP2	Nov 10–13	RAG pipeline · ChromaDB · RAGAS evaluation
TP3 + TP4	Nov 18	Multi-agent with CrewAI · intro to LangGraph
EDT 1–4	Nov 23 → Dec 8	Group project — agent on a real problem
Soutenance	Dec 9	Demo + Q&A
QCM	Dec 10	1h — key concepts from the module

References

- Wei et al. (2022) — Chain-of-Thought
  `arXiv:2201.11903`
- Yao et al. (2022) — ReAct
  `arXiv:2210.03629`
- LangChain Academy — Module 1
  `academy.langchain.com`
- OpenAI Cookbook
  `github.com/openai/openai-cookbook`

The thread — CM1 to TP1

- CM1 — *why* the loop works
- CM2 — *vocabulary* to code it
- TD1 — *build* it piece by piece
- TP1 — *assemble* the full ReAct agent

IMT Mines Alès
École Mines-Télécom

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

Komlan AYITE

komlan.ayite@mines-ales.fr

IMT Mines Alès / CERIS — Dept. 2IA

2ia